

Lab 06

Make and Environments Continued

Today's plan

01 Reading Checkpoint #5

The Reinhart-Rogoff Error

02 Makefiles

Review, variables, wildcards

03 Virtual Environments

Motivation, pip, conda, renv, and more

04 Make + Environments

Setting up envs via make

Reading Checkpoint #5

Make Continued

Make review

Target & Dependency Rules

Structure: target: dependencies. Recipe lines **MUST** start with a real **TAB** character.

Timestamp Rebuild Logic

Make checks file modification dates—only re-running steps if dependencies are newer than targets.

.PHONY Declarations

Declares non-file action targets (e.g., all, clean) to avoid filename conflicts.

Makefile Structure

Tab indent required

```
.PHONY: all clean report.pdf
all: report.pdf
clean_data.csv: raw_data.csv clean.py
    python3 clean.py
report.pdf: clean_data.csv report.Rmd
    Rscript -e "rmarkdown::render('report.Rmd')"
clean:
    rm -f clean_data.csv report.pdf
```

Managing Files Across Directories

Organized Project Layout

Separate source code (src/), raw input data (data/), and output artifacts (results/).

Explicit Path Dependencies

Targets and dependencies include relative directory paths so Make tracks exact locations.

Directory Creation (mkdir -p)

Always ensure target directories exist before running commands that output files.

Makefile (Directory Paths)

Relative paths

```
# Clean data from raw input
```

```
results/clean.csv: data/raw.csv src/clean.py
```

```
    mkdir -p results
```

```
    python3 src/clean.py data/raw.csv results/clean.csv
```

```
# Render report plot
```

```
results/figure.png: results/clean.csv src/plot.R
```

```
    mkdir -p results
```

```
    Rscript src/plot.R results/clean.csv results/figure.png
```

Simplifying Rules with Variables

Makefile Variables

`$(VAR)` & Automatic Vars

```
# User Variables
PYTHON = python3
SRC_DIR = src
DATA_DIR = data
OUT_DIR = results

# Rules using Variables & Automatic Shortcuts
$(OUT_DIR)/clean.csv: $(DATA_DIR)/raw.csv $(SRC_DIR)/clean.py
    mkdir -p $(OUT_DIR)
    $(PYTHON) $(SRC_DIR)/clean.py $< $@

# $< = first dependency | $@ = target name
```

User Variables (VAR = value)

Avoid hardcoding paths or executable names. Reference with `$(VAR)`.

Automatic Variable Shortcuts

- `$@`: Target filename
- `$<`: First dependency file
- `$$^`: All dependency files

Simplifying Rules with Variables

Makefile Variables

\$(VAR) & Automatic Vars

```
# User Variables
PYTHON = python3
SRC_DIR = src
DATA_DIR = data
OUT_DIR = results

# Rules using Variables & Automatic Shortcuts
results/clean.csv: data/raw.csv src/clean.py
    mkdir -p results
    python3 src/clean.py data/raw.csv src/clean.py results/clean.csv

# $< = first dependency | $@ = target name
```

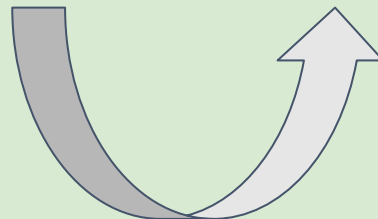
Makefile Variables

\$(VAR) & Automatic Vars

```
# User Variables
PYTHON = python3
SRC_DIR = src
DATA_DIR = data
OUT_DIR = results

# Rules using Variables & Automatic Shortcuts
$(OUT_DIR)/clean.csv: $(DATA_DIR)/raw.csv $(SRC_DIR)/clean.py
    mkdir -p $(OUT_DIR)
    $(PYTHON) $(SRC_DIR)/clean.py $< $@

# $< = first dependency | $@ = target name
```



Recap discussion

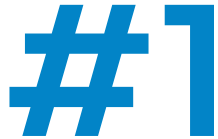
1. What is a phony target? When do you use them, and how?
2. What do `^``, `<`` and `@`` designate?
3. What happens if we modify the script we used to generate the data?
4. What does it mean to do a 'dry run' of your make file?
5. How do you execute a makefile? Does the name of the file need to be `Makefile``?

Lab part 1

Break

Virtual Environments

Motivation: The "Works on My Machine" Problem



Cause of Collaboration Friction

Hidden package version mismatches breaking shared code.

Version Drift Breaks Syntax

Code using Pandas 2.0 features crashes on a teammate's machine running Pandas 1.4.

Operating System Differences

C++ binary dependencies compiled on macOS fail when executed on Windows or Linux.

The Virtual Environment Solution

Isolated project libraries guarantee identical executions for everyone.

What is a Virtual Environment?



Isolated Package Directory

A self-contained folder storing exact package versions for one specific project.



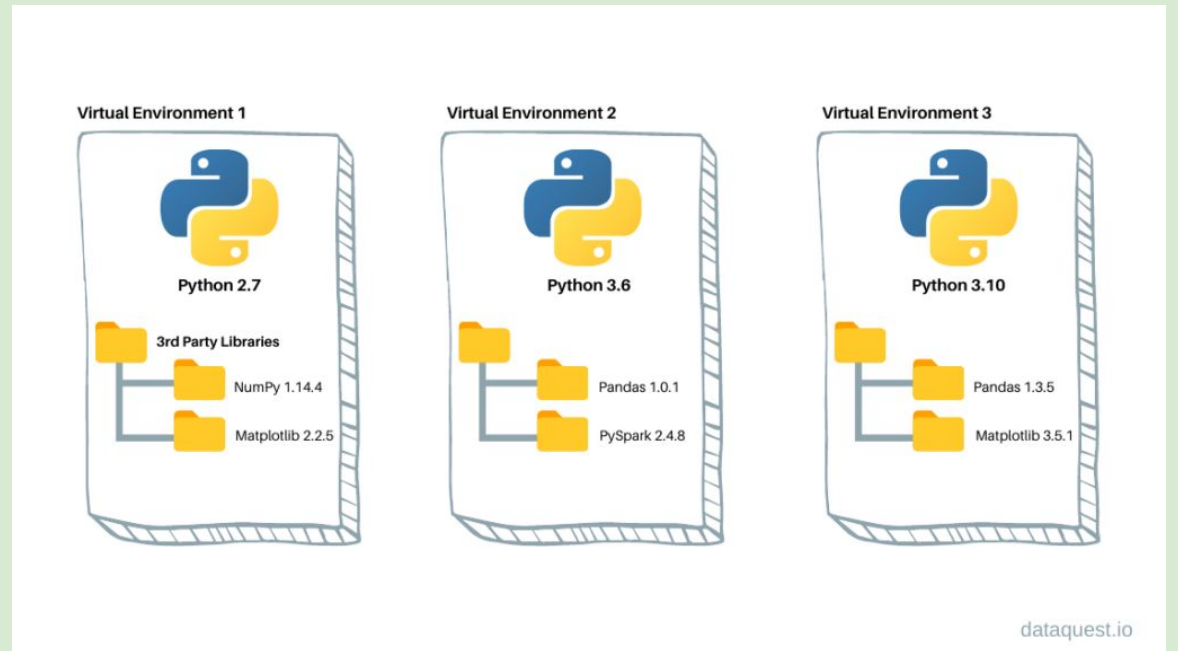
No Global Pollution

Prevents upgrades in Project A from breaking working scripts in Project B.



Declarative Lockfiles

Lightweight plain-text manifest files that document all exact dependencies.



Credit:

<https://www.dataquest.io/blog/a-complete-guide-to-python-virtual-environments/>

Python options

`venv`

Standard

Built-in lightweight Python manager.

```
python -m venv .venv  
pip install -r reqs.txt
```

Best for pure Python workflows.

`conda` / `mamba`

Cross-Lang

Handles C++, R, and Python packages.

```
conda env create -f environment.yml
```

Best for complex data science stacks.

`uv` / `pixi`

Modern

Extremely fast Rust-based tools.

```
uv venv  
uv pip sync lock.txt
```

Blazing fast dependency resolution.

R Reproducibility with `renv`

1

`renv::init()`

Initializes a project-local library independent of global R packages.

Project Setup

2

`renv::snapshot()`

Saves exact R package versions and repositories into `renv.lock`.

Creates `renv.lock`

3

`renv::restore()`

Restores exact packages on a teammate's machine using `renv.lock`.

Collaborator Sync

What to Track vs. What to Ignore

✓ ALWAYS Track in Git

Small text manifest files that define specifications.

`environment.yml`

`requirements.txt`

`renv.lock`

✗ Add to `.gitignore`

Heavy built binary libraries and local state.

`.venv/` or `venv/`

`renv/library/`

`.Rproj.user` / `.ipynb_checkpoints`

Lab part 2

Make + Environments

Automating Conda Envs with Make

One-Command Onboarding

Students or collaborators type `make env` to create and configure the entire environment automatically.

Syncing Dependencies

`make update-env` updates packages to match `environment.yml` whenever new tools are added.

Clean Teardown

`make clean-env` removes the local Conda environment cleanly without leaving orphan binaries.

Makefile (Conda Automation)

.PHONY Environment targets

```
ENV_NAME = ds-lab-env
REQ_FILE = environment.yml
.PHONY: env update-env clean-env
# Create Conda environment from spec file
env:
    conda env create -f $(REQ_FILE)
# Update environment if environment.yml changed
update-env:
    conda env update -f $(REQ_FILE) --prune
# Delete the environment
clean-env:
    conda env remove -n $(ENV_NAME)
```

Lab part 3